

Storage Migration Audit

A source-verified analysis of how Firebase Storage is wired through Event-Master, Admin-Event-Pro, and Event-Master-Web-Application — built to support a decision on moving off Firebase Storage without losing data or breaking any of the three apps.

Firestore project

event-master-2bf93 (shared by all 3 apps)

Bucket

event-master-2bf93.appspot.com (shared)

Backend

none — all 3 apps talk to Firebase directly

Security rules

not committed to any of the 3 repos — verify in console

Storage call sites

10 across all 3 apps (2 dead)

– Executive summary

- 1 The three apps' actual roles are inverted from the brief — verify this before planning further. Code shows `Event-Master` (mobile) is the consumer/booking app, not the entrepreneur app; `Admin-Event-Pro` is the entrepreneur self-service app (any self-signup gets full write access, no admin check found), not the platform-admin console; and *this* repo, `Event-Master-Web-Application`, is the admin console (event-template CRUD + vendor-application review), not the public browse/book site. See Part 1.
- 2 Firebase Storage is used in exactly 10 places across all 3 codebases (8 live, 2 dead code), each app reimplementing the same pick→upload→getDownloadURL→write-to-Firestore pattern independently, with no shared helper anywhere. See Part 2.
- 3 Every image field in every collection stores a full Firebase download URL (never a bare object key) — confirmed at all 9 live write sites and all read sites. This is good news for migration: the read/display layer already treats the field as an opaque string. See Part 6.
- 4 Seven distinct image flows exist: event-template images, vendor-template images, entrepreneur profile image (the closest analogue to a "company logo" — there is no separate company entity), a preset "cover image" (chosen from 6 bundled app assets, not freely uploaded), two separate gallery concepts (personal user gallery vs. vendor gallery), and a consumer event/booking photo. See Part 3.
- 5 Biggest migration risks: (a) Storage access is already blocked on the Spark plan, so even reading existing objects for a migration script may require briefly re-enabling billing; (b) no committed security rules mean current access control is unverifiable from source; (c) a path-construction bug double-nests the key for newly created event-template categories; (d) three independent orphan-leak bugs mean deleted/replaced images are never removed from Storage today.
- 6 Recommended target architecture: keep Firestore exactly as-is (storing an object key + provider-agnostic CDN URL instead of a raw Firebase URL), add a thin Node.js API that verifies Firebase Auth tokens and issues short-lived signed upload URLs, and let clients PUT directly to R2 behind Cloudflare's CDN. See Part 8.
- 7 Recommended storage provider: Cloudflare R2 — zero egress fees (all 3 apps are read-heavy image browsers), S3-compatible so any Node.js S3 SDK works, and Cloudflare's CDN sits in front of it for free. See Part 11.
- 8 Code areas to touch: 9 upload functions across 3 repos (list in Part 7), zero model-class changes (no typed models exist for images anywhere), zero UI/display changes (every widget already just renders a URL string).

- 9 **Recommended migration order:** Phase 0 discovery while Storage is still reachable → build the R2 + API + MediaService abstraction → migrate existing objects and Firestore fields → cut over the admin app first (smallest blast radius, one owner) → then the entrepreneur app → then the consumer app → remove `firebase_storage` last. See Part 9.
- 10 **What should NOT change:** Firestore itself, Firebase Auth, the collection/document structure, and the field *names* that already hold image URLs — only the value they hold and where the upload call goes.
- 11 **What should change first:** stop the 3 verified orphan-leak bugs and the path-double-nesting bug now, independent of any R2 decision — they are actively wasting Storage quota under the very Spark-plan constraint that triggered this audit.

01 Project architecture

Each of the three Flutter projects is a fully independent pub package — no shared package, no monorepo, no path dependency between them. The only thing that ties them together is a single shared Firebase project.

FRAMING CORRECTION

The brief describes Event-Master as the entrepreneur app, Admin-Event-Pro as the platform-admin console, and this web app as the public browse/book site. Reading the actual source shows the opposite mapping. Recommend confirming this with whoever wrote the original brief before using it to plan the migration, since Part 3 onward is built on the verified (not the assumed) mapping.

Event-Master mobile — actually the consumer / booking app

- **Stack:** Dart SDK `>=3.4.1 <4.0.0`, version `1.0.0+2` (`pubspec.yaml:20,22-23`).
`firebase_core ^4.11.0`, `firebase_auth ^6.5.4`, `cloud_firestore ^6.6.0`,
`firebase_storage ^13.4.3`.
- **State management:** `flutter_bloc ^9.1.1` is the real state layer; `get ^4.6.6` is used only for navigation, not as a state manager.
- **Architecture:** ad-hoc service classes under `lib/data_layer/services/*.dart` (`ClientProfile`, `MediaMethods`, `EventBookingMethods`, `DatabaseMethods`, `UserProfile`) call `Firestore.instance` / `Storage.instance` directly — no interfaces, no dependency injection, no repository layer. **No typed model classes exist for any image-bearing entity** — every Firestore document is consumed as a raw `Map<String, dynamic>` inside widget `build()` methods (e.g. `lib/presentation/components/dashboard/listview.dart:49`). The only typed model, `UserModel` (`lib/bussiness_layer.dart/entities/user_data.dart`), has no image fields.
- **Config:** single environment, project `event-master-2bf93`, bucket `event-master-2bf93.appspot.com` — no dev/staging/prod flavors, no `.env`.
- **Auth:** `ManageBloc` (`lib/data_layer/auth_bloc/manage_bloc.dart`) — email/password (checks `users/{uid}.platform == 'mobile'`, line 26), Google Sign-In (line 146), `SharedPreferences` session cache. Sign-up writes the password itself into Firestore (line 59) — flagged in Part 12.
- **Image packages:** `image_picker ^1.1.2`, `photo_view ^0.15.0`. No `cached_network_image` — every remote image is a plain `NetworkImage` / `Image.network`.

Admin-Event-Pro — actually the entrepreneur self-service app

- **Stack:** Dart SDK `>=3.3.4 <4.0.0`. Same Firebase package set as Event-Master, plus `firebase_app_check ^0.4.5`, `firebase_messaging ^16.4.1`.
- **Platforms:** android, ios, web, macos, windows all configured (linux throws `UnsupportedError` in `lib/firebase_options.dart:33-36`).

- **State management:** hybrid — `flutter_bloc ^9.1.1` (`ManageBloc` , `DashboardBloc` , `GeneratedBloc` , `ProfileBloc`) for real state, `get` for navigation/snackbars only.
- **Architecture:** `lib/data_layer/services/*.dart` service classes, but the layering leaks — `DashboardBloc._onUploadCoverImage` (`dashboard_bloc.dart:41-59`) calls `Storage` directly from a `Bloc`, and one upload flow's business logic lives inside a UI component file (`lib/presentation/components/generated_form/crud_add/submit_button.dart`). No model classes with `fromJson` / `toJson` exist anywhere in the repo.
- **Auth / role check:** no admin or role verification was found anywhere in the client code (a case-insensitive search for `isadmin` , `admins` , `customclaim` , `getIdTokenResult` , `role` across all of `lib/` returned zero matches). Sign-up (`manage_bloc.dart:59-64`) just writes to `entrepreneurs/{uid}` — the same collection every "vendor profile" service reads and writes — meaning any self-registered account is fully privileged in this app; whatever real gating exists must live entirely in Firebase Security Rules, which are not in this repo to review.
- **Image packages:** `image_picker ^1.1.2` , `cached_network_image ^3.4.0` (the only one of the 3 apps that caches images), `transparent_image ^2.0.1` .

Event-Master-Web-Application [this repo](#) — actually the admin console

- **Stack:** Dart SDK `>=3.4.1 <4.0.0` . Same Firebase package set. `file_picker ^8.0.4` instead of `image_picker` (appropriate for a web target). No HTTP client package, no Node/backend folder anywhere in the repo.
- **Architecture:** `lib/bussiness_layer` (models + UI-helper "repos"), `lib/data_layer` (`auth_bloc` , `category_bloc` , `services/*` — `DatabaseMethods` / `SubDatabaseMethods`), `lib/presentation_layer` (`components` , `screens`). Layering is nominal — widgets instantiate `DatabaseMethods()` directly (`lib/presentation_layer/components/form/template_carousal.dart:18`), no interfaces or DI.
- **Auth / role gate:** email/password only; access is gated by a `developer` Firestore document with `platform == 'web'` (`lib/data_layer/auth_bloc/auth_bloc.dart:29-43`) — a Firestore field check, not a Firebase Auth custom claim. The cached session in `shared_preferences` is trusted without re-validation on relaunch (`auth_bloc.dart:91-121`).
- **Config:** same shared project/bucket. No `.firebaserc` , no `storage.rules` / `firestore.rules` committed. `web/index.html` has no embedded Firebase JS config (uses FlutterFire's generated `firebase_options.dart` instead).
- **What this app actually does:** Category/SubCategory (event-template) CRUD, plus a vendor-application review workflow (Requests / Accepted / Rejected tabs). There is no cart, checkout, or booking model anywhere in this repo.

How the three communicate

They don't talk to each other directly — there is no inter-app API, no shared package, no message queue. All communication is **indirect, through the shared Firestore project and Storage bucket**: whichever app writes a document (or an image URL into that document) is read by whichever other app queries the same collection. Every app authenticates independently against the same Firebase Auth tenant with its own role-check convention (a `platform` field, or none at all) — there is no single source of truth for "who is an admin" across the three apps.

02 Firebase Storage usage map

Ten places in the three codebases call the `firebase_storage` SDK directly. No project has a shared `StorageService` / `MediaService` wrapper — each site re-implements pick → upload → `getDownloadURL()` → write-to-Firestore

independently, with small inconsistencies between them (different "is this already a URL?" guards, different path shapes, different error handling).

APP	FILE : LINE	CLASS.METHOD	OPERATION	IMAGE TYPE
Event-Master	<code>data_layer/services/prifile.dart:7-58</code>	<code>ClientProfile.updateProfile</code>	upload + getURL	User profile picture
Event-Master	<code>data_layer/services/medias.dart:17-34</code>	<code>MediaMethods.uploadImages</code>	upload + getURL	Personal gallery photo
Event-Master	<code>data_layer/services/medias.dart:63-70</code>	<code>MediaMethods.deleteImage</code>	delete (Firestore only — Storage object never removed)	Personal gallery photo
Event-Master	<code>data_layer/services/create_event.dart:107-129</code>	<code>EventBookingMethods.uploadImage</code>	upload + getURL	Event / booking photo
Admin-Event-Pro	<code>data_layer/services/profile.dart:95-111</code>	<code>UserProfile.uploadImageToFirebase</code>	upload + getURL	Entrepreneu profile + gallery
Admin-Event-Pro	<code>bussiness_layer/repos/upload_imgae_file.dart:5-18</code>	<code>uploadFile (dead code)</code>	upload + getURL, never called	—
Admin-Event-Pro	<code>data_layer/services/generated_vendor.dart:76-91</code>	<code>GeneratedVendor.uploadImageToFirebase</code>	upload + getURL (re-run on edit, old blob never deleted)	Vendor template image
Admin-Event-Pro	<code>data_layer/services/cover_images.dart:6-32</code>	<code>CoverImageMethods.uploadImage</code>	upload (of a bundled app asset) + getURL	Preset cover image
Web-Application	<code>data_layer/services/category.dart:85-101</code>	<code>DatabaseMethods.uploadImage</code>	upload + getURL (edit re- runs, old blob never deleted)	Event- template cover image
Web-Application	<code>data_layer/services/sub_category.dart:57-73</code>	<code>SubDatabaseMethods.uploadImage</code>	upload + getURL (edit re- runs, old blob never deleted)	Sub-template image
Web-Application	<code>data_layer/category_bloc/vendor_category_bloc.dart:46-56</code>	<code>VendorCategoryBloc.uploadImageEvent (dead code)</code>	upload, event never dispatched	—

Every live upload site resolves a permanent HTTPS download URL via `getDownloadURL()` (never a bare path) and writes that string straight into a Firestore field — no code anywhere parses that URL back into a Firebase `Reference` (no `refFromURL()` calls exist), so nothing depends on Firebase's specific URL shape beyond "fetch it and render it."

03 Media lifecycles, traced from the code

Seven distinct image flows were found. None matches a generic "admin uploads → Storage → Firestore → user app" template exactly — each has its own quirks, traced below.

A · Event-template images

1. Uploaded by the logged-in `developer` account inside `Web-Application` (this repo) — the admin console, not this app's public site.
2. Via `DatabaseMethods.uploadImage / SubDatabaseMethods.uploadImage`
3. Path: `category_images/{categoryId}/` — but a bug in `category_submit_b.dart:54` double-nests it to `category_images/{id}/Categories` for newly created categories only; edits and all sub-categories get the clean path.
4. URL stored in `Categories/{id}.imagePath` or `Categories/{id}/SubCategories`
5. Read by `Event-Master` mobile (`listview.dart:49,88 ; sub_templates.dart:83,134-141`) and by `Web-Application`'s own list/detail screens; `Admin-Event-Pro` reads it too (`listof_templates.dart:85,139` read-only).
6. Displayed via `NetworkImage / Image.network`, with a `startsWith('http')` check falling back to a bundled placeholder asset.
7. Editing re-uploads and overwrites the URL field but never deletes the previous Storage object — confirmed orphan leak on every edit.

B · Vendor-template images

1. Uploaded by the entrepreneur, inside `Admin-Event-Pro`.
2. Via `GeneratedVendor.uploadImageTo` called from `addGeneratedCategoryDetail / updateGeneratedCategoryDetail`
3. Path: `generated_images/{millis}` — flat, no id/uid scoping, no file extension preserved.
4. URL stored in `entrepreneurs/{uid}/vendorDetails` and `.images[].imageUrl`.
5. Read by `Event-Master` mobile (`vendor_list.dart:95,129 ; event_detail.dart:201-204`) and by `Web-Application`'s review tabs (`request_tab.dart , accepted.dart , rejected.dart , vendor_detail.dart:54-56,126-`)
6. Displayed via plain `NetworkImage`.
7. Update re-uploads and overwrites the field; the previous object is never deleted.

C · "Company logo"

1. There is no separate "company" entity in the data model. The closest analogue is `entrepreneurs/{uid}.profileImage`
2. Uploaded by the entrepreneur in `Admin-Event-Pro` via `UserProfile.uploadImageToFirestore`
3. Path: `profile_images/{millis}_{filename}`
4. Stored/read/displayed identically to flow B's profile field.

D · "Company cover image"

1. This is not a free upload — it's a pick from 6 fixed, bundled app assets (`Assigns.image1..6 , lib/common/assigns.dart:51-56`) inside `Admin-Event-Pro`.
2. Via `CoverImageMethods.uploadImage` called directly from `DashboardBloc._onUploadCoverImage` (Storage call made from a Bloc, not a service).
3. Path: `cover_images/{assetFilename}` — the chosen bundled asset gets re-uploaded to Storage under its own filename.
4. URL stored in `entrepreneurs/{uid}/coverImage`
5. Because the source set is fixed (6 assets), these 6 objects could simply become 6 static CDN files post-migration rather than per-user migrated objects.

E • Gallery images (two separate concepts)

1. Personal user gallery — `Event-Master` mobile, `MediaMethods.uploadImages`, path `uploads/{millis}.jpg`, stored at `users/{uid}/medias/{id}.url`. Delete removes the Firestore doc only — the Storage blob is never removed (`medias.dart:63-70`).
2. Vendor/entrepreneur gallery — the `.images[]` arrays described in flows B and C, uploaded by the same two Admin-Event-Pro services.

F • User profile image

1. Uploaded by the consumer in `Event-Master` mobile via `ClientProfile.updateProfile`.
2. Path: `client_profile/{fileName}` — reuses the original picked-file basename in a flat, shared (non-uid-scoped) folder.
3. URL stored in `users/{uid}.imageUrl`; displayed via `NetworkImage` in `profile.dart`, `stack_appbar.dart`.

G • Event / booking photo

1. Uploaded by the consumer in `Event-Master` mobile via `EventBookingMethods.uploadImage`, called from both `addEvent` and `updateEvent`.
2. Path: `event_images/{millis}.jpg`.
3. URL stored in `users/{uid}/events/{id}.imageUrl`; displayed in `events.dart`, `event_detail.dart`.

04 Firestore data model — image fields

No project defines a typed model class for any image-bearing document — every read/write goes through raw `Map<String, dynamic>`. The table below is the complete, union-of-all-three-apps set of image fields found.

COLLECTION	FIELD	CURRENT VALUE TYPE	WRITTEN BY	READ BY
<code>users/{uid}</code>	<code>imageUrl</code>	Firebase download URL	Event-Master	Event-Master
<code>users/{uid}/medias/{id}</code>	<code>url</code>	Firebase download URL	Event-Master	Event-Master
<code>users/{uid}/events/{id}</code>	<code>imageUrl</code>	Firebase download URL	Event-Master	Event-Master
<code>entrepreneurs/{uid}</code>	<code>profileImage</code>	Firebase download URL	Admin-Event-Pro	Event-Master, Admin-Event-Pro, Web-Application
<code>entrepreneurs/{uid}</code>	<code>images[].image</code>	Firebase download URL	Admin-Event-Pro	Web-Application
<code>entrepreneurs/{uid}/vendorDetails/{id}</code>	<code>imageUrl</code>	Firebase download URL	Admin-Event-Pro	Event-Master, Web-Application
<code>entrepreneurs/{uid}/vendorDetails/{id}</code>	<code>images[].imageUrl</code>	Firebase download URL	Admin-Event-Pro	Event-Master, Web-Application
<code>entrepreneurs/{uid}/coverImages</code>	<code>selectedImage.url</code>	Firebase download URL (of a bundled asset)	Admin-Event-Pro	Admin-Event-Pro
<code>Categories/{id}</code>	<code>imageUrl</code>	Firebase download URL	Web-Application	Event-Master, Admin-Event-Pro, Web-Application
<code>Categories/{id}/SubCategories/{id}</code>	<code>imageUrl</code>	Firebase download URL	Web-Application	Event-Master, Admin-Event-Pro, Web-Application
<code>Categories/{id}/SubCategories/{id}</code>	<code>image</code>	transient pre-upload filename (not a URL)	Web-Application	not re-read for display

Example, as requested: collection `Categories`, document e.g. `wedding_x7f2`, field `imageUrl`, current value a string like

`https://firebasestorage.googleapis.com/v0/b/event-master-2bf93.appspot.com/o/category_images%2Fwedding_x7f2%2Fcover.jpg?alt=media&token=...`

No collection literally named `companies` or `bookings` exists — "bookings" in the brief corresponds to `users/{uid}/events`, and "company" corresponds to the

05 Storage path structure — as actually implemented

Nine distinct top-level prefixes are in use, with no shared naming convention, no consistent uid/tenant scoping, and inconsistent filename strategies (some hashed-by-timestamp, some by original filename, most without a preserved extension).

```
event-master-2bf93.appspot.com/
├─ category_images/                – Web-Application (event templates)
│   └─ {categoryId}/
│       └─ {imageName}              Category cover, category.dart:90
│       └─ Categories/{categoryId}/{imageName}  ▲ double-nested – new categories only, category_submit_b.dart:54
│   └─ {subCategoryId}/{imageName}  SubCategory image, sub_category.dart:62
├─ profile_images/                 – Admin-Event-Pro
│   └─ {millis}_{filename}          Entrepreneur profile + gallery, profile.dart:97-98
├─ generated_images/               – Admin-Event-Pro
│   └─ {millis}                    Vendor-template image (main + gallery loop), generated_vendor.dart:78-80 – no extension
├─ generatedVendors/               – Admin-Event-Pro, dead code, upload_imgae_file.dart, never called
│   └─ {filename}
├─ cover_images/                   – Admin-Event-Pro
│   └─ {assetFilename}              One of 6 fixed bundled app assets, cover_images.dart:12-14
├─ client_profile/                 – Event-Master mobile
│   └─ {fileName}                   Consumer profile picture, prifile.dart:32 – original filename, no uid scope
├─ uploads/                         – Event-Master mobile
│   └─ {millis}.jpg                 Consumer personal gallery, medias.dart:23
└─ event_images/                   – Event-Master mobile
    └─ {millis}.jpg                 Consumer event/booking photo, create_event.dart:112-113
```

06 Image URL dependencies — URL vs. key

Answer, verified at all 9 live write sites and every read site across all three apps: **option A only** — the system stores full Firebase Storage download URLs (`https://firebasestorage.googleapis.com/...`), never a bare object path/key, and never both.

Every write site resolves the URL via `getDownloadURL()` before touching Firestore. Every read site's only "is this real?" logic is a string check — `field.startsWith('http') ? NetworkImage(field) : AssetImage(field)` — appearing independently in all three apps (e.g. `listview.dart:49` in Event-Master, `generated_vendor.dart:25,150` in Admin-Event-Pro, `carousal.dart:60-62` and `vendor_detail.dart:54-56` in Web-Application). No code anywhere calls `FirebaseStorage.refFromURL()` or otherwise parses the URL back into a Firebase-specific reference — the value is treated purely as an opaque string to hand to a network-image widget.

Practical implication: nothing in any of the three apps assumes the URL is specifically a Firebase URL beyond "it starts with `http`." This is the single most migration-friendly fact in the whole system — a Cloudflare R2/CDN URL that also starts with `https://` will satisfy every existing check with zero UI changes.

07 Firebase Storage → Cloudflare R2 — migration impact

QUESTION	ANSWER
Code locations to change	The internals of the 8 live upload functions listed in Part 2 (swap the Firebase Storage calls for an R2/API call); zero changes needed to any calling code, since callers already just <code>await</code> a URL string back.
Services/classes to replace	<code>ClientProfile.updateProfile</code> , <code>MediaMethods.uploadImages / deleteImage</code> , <code>EventBookingMethods.uploadImage (Event-Master)</code> , <code>UserProfile.uploadImageToFirebase</code> , <code>GeneratedVendor.uploadImageToFirebase</code> , <code>CoverImageMethods.uploadImage (Admin-Event-Pro; delete the dead <code>upload_imgae_file.dart</code> outright)</code> , <code>DatabaseMethods.uploadImage</code> , <code>SubDatabaseMethods.uploadImage (Web-Application)</code> .
Models needing changes	None — no typed image models exist anywhere to change.
Firestore fields to migrate	The 10 fields listed in Part 4's table (value changes only — same field names, same collections).
Can old Firebase URLs stay temporarily?	Yes, functionally — every read site already treats the field as an opaque URL, so old and new URLs can coexist field-by-field during a staged rollout. The catch: they only keep working as long as the Storage bucket itself keeps serving traffic, which is exactly what's currently blocked on the Spark plan — so "temporarily" is bounded by however long Storage stays reachable, not indefinite.
Introduce a <code>MediaService</code> abstraction?	Yes, recommended — it's the fix for the fact that this pattern is already duplicated 9 times with small inconsistencies. Since the 3 Flutter apps share no package today, the cleanest place for this abstraction is a new backend API (see Part 8) rather than a shared Dart package across 3 independently-versioned pubspecs.
Migrate without changing business logic?	Yes — <code>addEvent</code> , <code>addGeneratedCategoryDetail</code> , <code>addVendorCategoryDetail</code> , etc. don't need to change; only what happens inside their upload helper changes.
Can Admin/Entrepreneur/User apps share one media architecture?	Architecturally yes — all three already reinvented the identical 3-step pattern independently. Practically, share it by centralizing the actual upload/delete logic in the new API rather than trying to share Dart code across 3 separate pub packages.

08 Recommended target architecture

Evaluated against what actually exists — no backend today, three independent Flutter clients, a single shared Firebase project with no committed security rules, and two verified gaps in server-side authorization (Admin-Event-Pro's self-signup, Web-Application's Firestore-flag role gate). The architecture below is shaped to close those specific gaps, not a generic template.

- **Authentication** stays exactly where it is — Firebase Auth, already wired into all 3 apps. What changes is that the new API verifies the Firebase ID token server-side (via the Admin SDK) on every request, rather than any app trusting a Firestore `platform` field or an un-gated `entrepreneurs/{uid}` write.
- **Authorization** moves server-side, in the API: before issuing a signed upload/delete URL for a `entrepreneurs/{uid}/...` key, the API checks the token's uid matches `{uid}`; before issuing one for `category_images/...`, it checks a real admin custom claim (not a Firestore document flag).
- **Who generates upload URLs:** the API, using an S3-compatible SDK against R2, issuing short-lived presigned PUT URLs scoped to one exact object key.
- **Signed URLs are necessary:** yes — the R2 bucket itself should be fully private; the only public surface is a CDN domain in front of it.
- **Direct client → R2 upload:** yes, for the PUT itself (keeps bandwidth off the API server) — but issuing the signed URL and confirming/writing the Firestore field both go through the API, so authorization is never skipped.
- **Deletion:** always API-mediated. This is also the fix for the three verified orphan-leak bugs — the API deletes the previous object by key before (or atomically with) writing the new one.
- **Replacement:** delete-then-upload, wrapped as one API call so Firestore is only updated once both storage operations succeed.

- **Public vs. private:** everything found in this audit (templates, vendor photos, profile pictures) is catalog/marketing content meant to be browsed publicly — recommend all-public-read via the CDN, with only the write side (the presigned PUT and the delete call) gated.
- **URL generation:** store the R2 *object key* in Firestore, not a full URL, and construct the display URL by concatenating a stable CDN base domain — this is the one real schema change, and it directly fixes the coupling found in Part 6 (nothing today stores a bare key).
- **CDN:** Cloudflare's own edge sits in front of R2 natively — no extra service to stand up.
- **Preventing unauthorized uploads:** a signed URL scoped to one key, short TTL, issued only after the ownership/role check above.
- **Preventing cross-tenant deletes:** the API checks that the authenticated uid owns the entrepreneur/admin document referencing a key before it will delete that key — closing the exact gap that today lets any self-signed-up Admin-Event-Pro account write anywhere in `entrepreneurs/`.
- **Orphaned images:** once creation/deletion only happens through the API, add a simple later reconciliation job (list R2 keys, diff against Firestore-referenced keys) — not urgent given the small image volumes implied by this codebase, but worth having before volume grows.

FIT CHECK

The "client → Node API → R2 → CDN" shape fits this codebase well specifically *because* two of the three apps currently have no real server-side authorization at all — adding a thin API is what actually closes those gaps, not just a storage-provider swap. It would *not* be worth the added complexity if the apps already had solid Firestore/Storage Security Rules doing this job — but those rules aren't in any of the three repos to verify, so building the check into a new API is the safer default.

09 Migration strategy

Phase 0	Backup and discovery	
	CHANGES	Nothing in the apps yet. Enumerate every object in the Firebase bucket and every Firestore doc holding one of the 10 fields from Part 4, while Storage is still reachable (may require briefly re-enabling Blaze billing — see Part 10).
	PROJECT	None of the 3 client apps; a one-off script using the Firebase Admin SDK.
	RISK	If Storage stays inaccessible, this phase blocks everything downstream — it's the one true prerequisite.
	ROLLBACK	Read-only phase, nothing to roll back.
Phase 1	Create new storage architecture	
	CHANGES	Provision the R2 bucket, custom/CDN domain, and the new Node.js API skeleton with Firebase Admin SDK token verification.
	PROJECT	New repo/service — none of the 3 existing ones change yet.
	RISK	Low — additive only.
	ROLLBACK	Delete the new service; nothing else depends on it yet.
Phase 2	Build the MediaService abstraction	
	CHANGES	Implement upload/delete/replace endpoints in the API, matching the shape of the 8 functions in Part 7 so each can be swapped in with a matching signature.
	PROJECT	New API service.
	RISK	Low — still not called by any client.
	ROLLBACK	N/A, not yet wired in.

Phase 3**Migrate existing Firebase images**

CHANGES	Copy every object discovered in Phase 0 into R2 under a clean key scheme (fixing the double-nested category-image bug and the missing-extension keys along the way).
PROJECT	One-off script, no app changes.
RISK	Data loss if the mapping (Part 10) misses an object — verify counts before Phase 4.
ROLLBACK	R2 objects can simply be deleted; source Firebase objects are untouched.

Phase 4**Migrate Firestore image references**

CHANGES	Batch-update the 10 fields from Part 4 to the new CDN URL/key. Keep the old Firebase URL alongside temporarily if useful for a diff/verification pass.
PROJECT	One-off script against Firestore; no app code changes yet.
RISK	Medium — a partial batch failure could leave some docs on old URLs and some on new; run idempotently and verify a sample.
ROLLBACK	Restore the field from the Phase-0 backup export.

Phase 5**Update Admin app (this repo)**

CHANGES	Point <code>DatabaseMethods.uploadImage</code> / <code>SubDatabaseMethods.uploadImage</code> at the new API instead of <code>FirebaseStorage.instance</code> ; fix the double-nested path bug as part of this rewrite since it's the same code being touched.
PROJECT	Event-Master-Web-Application.
RISK	Lowest blast radius of the three — single admin user base, smallest write surface (2 functions).
ROLLBACK	Revert to the old service class; Firestore fields still hold valid URLs either way.

Phase 6**Update Entrepreneur app**

CHANGES	Replace the 3 live upload functions in Admin-Event-Pro; delete the dead <code>upload_imgae_file.dart</code> ; move the Storage call out of <code>DashboardBloc._onUploadCoverImage</code> and into the new client.
PROJECT	Admin-Event-Pro.
RISK	Medium — 3 functions, more entry points, real users depend on it daily.
ROLLBACK	Same class-level revert as Phase 5.

Phase 7**Update User app**

CHANGES	Replace the 3 live upload functions in Event-Master mobile; fix the two orphan-leak bugs (<code>medias.dart</code> delete, profile/event replace) as part of the same rewrite.
PROJECT	Event-Master.
RISK	Highest of the three — largest, most active user base; ship behind a feature flag if possible.
ROLLBACK	Same class-level revert.

Phase 8**Testing**

CHANGES	Upload/delete/replace through each of the 7 flows in Part 3, on each of the 3 apps; verify no orphan accumulates in R2; verify Firestore fields display correctly on all 3 clients simultaneously.
PROJECT	All three.
RISK	A missed flow (e.g. the preset cover-image picker in flow D) is easy to overlook since it behaves differently from a normal upload.
ROLLBACK	N/A — verification phase.

Phase 9**Remove Firebase Storage dependency**

CHANGES	Drop <code>firebase_storage</code> from all 3 <code>pubspec.yaml</code> files; decommission the Firebase bucket.
PROJECT	All three.
RISK	Irreversible once the bucket is deleted — keep it (even on a downgraded/idle state) for one full release cycle after cutover before deleting anything.
ROLLBACK	None past this point — this is the point of no return; gate it on Phase 8 passing cleanly for all 7 flows.

10 Existing Firebase images — preserving them

- Types of existing images: the 7 flows in Part 3 map to roughly 6 distinct object shapes in the bucket (event-template covers, sub-template images, vendor-template images +

gallery, entrepreneur profile + gallery, the 6 fixed cover-image assets, personal user gallery photos, event/booking photos).

- **How Firestore references them:** exactly the 10 fields listed in Part 4 — always a full download URL, never a bare key (Part 6).
- **Can the same logical image identity be preserved?** Yes — since each Firestore field already holds exactly one URL per logical image, migration is a 1:1 field overwrite once the object is copied; no schema change is required to preserve identity, only to store a key/CDN-URL going forward.
- **What's needed to migrate them:** (a) admin-level read access to the Storage bucket (currently blocked — see the callout below), (b) an export of every Firestore document containing one of the 10 fields, (c) a stable target key scheme in R2.
- **Can filenames/paths be extracted from current URLs?** Yes — a Firebase download URL encodes the object path in a URL-encoded segment between `/o/` and the `?alt=media&token=...` query string (e.g. `category_images%2FabcId%2Fcover.jpg` decodes to `category_images/abcId/cover.jpg`) — this is a straightforward, scriptable decode.
- **Can a script auto-map Firebase object → R2 object → Firestore reference?** Yes, in three steps: (1) decode every stored URL back to its bucket object path, (2) copy that object's bytes to the same (or a cleaned-up) key in R2, (3) batch-write the new key/CDN URL back into the exact Firestore field it came from — since Part 6 confirmed every field holds exactly one opaque URL string, this mapping has no ambiguous cases to resolve.

THE BILLING TENSION

Reading the actual objects for Phase 0/3 requires the bucket to be reachable, which is exactly what's currently blocked on the Spark plan. Practically, the Blaze (pay-as-you-go) plan required to unlock it still has a free tier (5 GB stored, 1 GB/day served, at the time of writing) — given the small, single-project image footprint found across all three apps in this audit, briefly enabling Blaze just long enough to run the export script, then either downgrading or simply staying within the free tier afterward, is likely far cheaper than "permanently paying a large amount." Confirm current Firebase pricing before committing, since it changes over time.

11 Provider comparison, for this specific system

	FIREBASE STORAGE	CLOUDFLARE R2	CLOUDINARY	SUPABASE STORAGE
Storage cost	~\$0.026/GB/mo	\$0.015/GB/mo	Higher, bundled with transform credits	~\$0.021/GB/mo
Bandwidth/egress	Paid egress	Zero egress fees	Paid, generous free tier	Paid egress
Built-in CDN	Via Firebase Hosting rewrite only	Native, Cloudflare edge	Native + transforms	Basic
Image transforms	None built-in	None built-in (pair with Cloudflare Images if needed)	Best-in-class	Basic resize
Flutter integration	Native SDK (what's used today)	S3-compatible SDKs, or plain HTTPS PUT to a signed URL	REST/SDK	SDK, tied to Supabase Auth
Node.js integration	Admin SDK	Any S3 SDK (aws-sdk, @aws-sdk/client-s3)	Official SDK	Official SDK
Signed URLs	Token-in-URL, long-lived	Standard S3 presigned URLs, short TTL	Signed upload presets	Signed URLs supported
Migration difficulty	—	Low — S3-compatible, and this codebase already treats every field as an opaque URL (Part 6)	Low-medium — different upload model (unsigned presets)	Medium — pulls in Supabase Auth/DB too
Vendor lock-in	High (proprietary SDK + URL format)	Low (S3-compatible, portable)	Medium (transform URLs are Cloudinary-specific)	Medium (bundled platform)
Fit for 3 independent read-heavy Flutter clients	Current, but blocked on Spark	Good — zero egress matters most for a browse-heavy catalog app	Good if image transforms (thumbnails) are wanted without building them	Good only if also adopting Supabase for auth/DB, which isn't the case here

Recommendation: Cloudflare R2, for three reasons specific to this codebase: (1) zero egress cost matters most here because all three apps are read-heavy image browsers, not upload-heavy; (2) it's S3-compatible, so the new Node.js API in Part 8 can use any mainstream S3 SDK rather than a bespoke client; (3) Part 6 already confirmed nothing in any app depends on Firebase's specific URL format, so there's no hidden coupling cost to swapping providers. The one gap R2 doesn't cover — on-the-fly thumbnails/compression, which Part 12 notes is missing everywhere today — can be added later via Cloudflare Images or a simple resize step in the new API, without changing the recommendation.

12 Problems found in the current implementation

Critical

Firebase Storage is currently inaccessible on the Spark plan
Blocks all 8 live upload paths and any new-object reads system-wide. This is the triggering issue for the whole audit and gates Phase 0/3 of any migration.

Critical

No server-side authorization anywhere in Admin-Event-Pro
Self sign-up grants full write access to `entrepreneurs/{uid}` and everything under it — no admin/role check found anywhere in the client (`manage_bloc.dart`). Whatever protection exists depends entirely on unreviewed Firebase Security Rules.

Critical

Plaintext passwords written to Firestore, in all three apps
`manage_bloc.dart:59` (Event-Master), `manage_bloc.dart:62` (Admin-Event-Pro), `auth_bloc.dart:66` (Web-Application) — unrelated to Storage, but a serious finding surfaced during this audit and worth fixing independently.

High Double-nested storage path for newly created event-template categories

`category_submit_b.dart:54` — new categories get `category_images/{id}/Categories/{id}/{fileName}`, edited categories and all sub-categories get the clean path. A migration mapping (Part 10) must handle both shapes.

High Orphaned Storage objects accumulate on every edit and delete

Confirmed in three independent places: `medias.dart` delete (Storage object never removed), `generated_vendor.dart` update (old blob kept), and `category.dart` / `sub_category.dart` edit (old blob kept). This is actively wasting Storage quota under the exact plan constraint that triggered this audit.

High No Firestore/Storage Security Rules committed to any of the 3 repos

Current access control can't be verified from source — it must be audited directly in the Firebase console before any migration or hardening decision.

High Web-Application trusts a cached session without re-validation

`auth_bloc.dart:91-121` — a stale or revoked session can still pass the client-side check on relaunch.

Medium Nine near-identical, duplicated upload implementations, no shared service

Each of the 3 apps reinvented the same pick→upload→getURL→write pattern independently, with small, inconsistent guards (`Uri.parse().isAbsolute` vs. `startsWith('http')`).

Medium Inconsistent, non-uid-scoped paths with filename-collision risk

`client_profile/$fileName` reuses the picked file's original basename in a flat shared folder; `generated_images/$millis` drops the file extension entirely.

Medium No image compression, resizing, or thumbnailing anywhere

Full-resolution images are uploaded and displayed even in list/grid views across all three apps.

Medium No image caching in two of the three apps

`cached_network_image` is only a dependency in Admin-Event-Pro; Event-Master and Web-Application re-fetch full images on every rebuild.

Medium Dead code

`upload_imgae_file.dart` (Admin-Event-Pro) and `VendorCategoryBloc.uploadImageEvent` (Web-Application) are unreachable but still call the Storage SDK — safe to delete, but worth confirming nothing else references them first.

Medium Weak error handling around uploads

Bare `print` / `catch` blocks with no user-facing partial-failure signal in the multi-image upload loops.

Low Hardcoded bucket string repeated across platform blocks

`firebase_options.dart` repeats `event-master-2bf93.appspot.com` across 5 platform configurations in both Event-Master and Admin-Event-Pro — not a secret, just duplication.

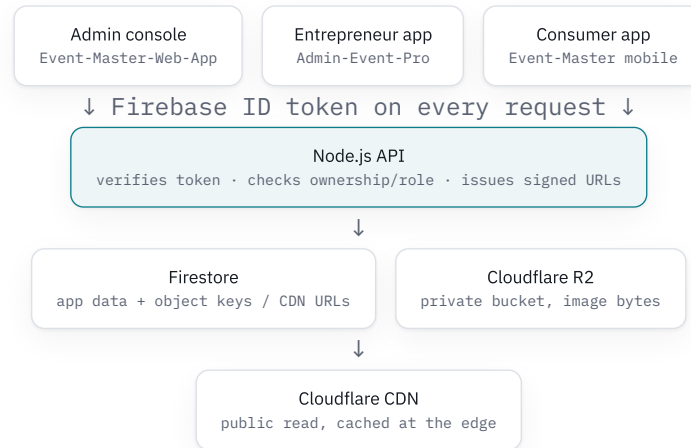
Low Unused dependency

`googleapis_auth` is declared in Admin-Event-Pro's `pubspec.yaml` but never imported anywhere.

Low Duplicated UI widgets, unrelated to storage

`entrepreneur_list.dart` / `user_list.dart`, and the Accepted/Rejected/Requests review tabs in Web-Application, are near-identical — a code-quality note, not a migration blocker.

13 Final architecture



- The three Flutter apps keep using Firebase Auth exactly as today; every call that touches media now goes through the API instead of the Firebase Storage SDK directly.
- The Node.js API is the one new component — it verifies the caller's identity and ownership/role, then either issues a short-lived signed R2 upload URL or performs a delete, and is the only thing that writes the resulting key/URL into Firestore.
- Firestore keeps its existing collections and field names (Part 4) unchanged; only the value in each image field changes, from a Firebase download URL to an R2 object key or CDN URL.
- R2 stays private; clients never address it directly except via the signed PUT URL the API hands them.
- The CDN is the only public-facing surface for reads — every display widget in all three apps already just renders a URL string (Part 6), so this swap is invisible to the UI layer.